

Python Programming

Day 10: Project Preparation & Integration

Main Guard, Modules, Data Pipeline, Error Handling, Project Structure

Contents

1	학습 목표	2
2	Main Guard와 프로그램 진입점	2
2.1	최종 코드: 01_main_guard.py	2
3	모듈 구조	3
3.1	모듈 코드: calculator.py	3
3.2	최종 코드: 02_module_structure.py	3
4	Data Pipeline	4
4.1	최종 코드: 03_data_pipeline.py	4
5	Error Handling Integration	4
5.1	최종 코드: 04_error_handling.py	5
6	Project Structure	5
6.1	데이터 모듈: data.py	6
6.2	분석 모듈: analysis.py	6
6.3	최종 코드: 05_main.py	6
7	Day 10 종합 실습: Student Score Analyzer	7
7.1	프로그램 요구사항	7
7.2	최종 코드: practice_day10.py	7
7.3	프로그램 구조 분석	8
8	실습 파일 목록	8
9	핵심 요약	8

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- `if __name__ == "__main__":`의 목적 설명하기
- `main()` 함수를 이용해 프로그램의 실행 흐름 구성하기
- Python 파일을 모듈로 분리하고 `import`로 불러오기
- 입력, 처리, 분석, 출력의 데이터 흐름을 함수로 구성하기
- 함수로 나눈 프로그램에 예외 처리를 통합하기
- 역할별 파일을 분리해 작은 프로젝트 구조 설계하기
- Day 10의 개념을 하나의 프로그램에 통합하기

2. Main Guard와 프로그램 진입점

Python 파일은 직접 실행할 수도 있고, 다른 파일에서 모듈로 불러와 사용할 수도 있다. 이 두 상황을 구분할 때 사용하는 것이 `__name__`과 `__main__`이다.

파일을 직접 실행하면 Python은 해당 파일의 `__name__` 변수에 `"__main__"`을 저장한다. 반대로 다른 파일에서 `import`하면 `__name__`에는 모듈 이름이 들어간다.

따라서 다음 조건문은 현재 파일이 직접 실행된 경우에만 내부 코드를 실행한다.

```
1 if __name__ == "__main__":
2     main()
```

`main()`이라는 이름 자체는 Python이 강제하는 특별한 이름이 아니다. 다른 이름을 사용해도 실행에는 문제가 없다. 다만 프로그램의 주 실행 흐름을 담당하는 함수라는 의미가 명확하기 때문에 관례적으로 `main()`을 많이 사용한다.

2.1. 최종 코드: 01_main_guard.py

```
1 def greet(name):
2     print(f"Hello, {name}!")
3
4
5 def main():
6     print("Program started")
7     greet("Alice")
8     print("Program finished")
9
10
11 if __name__ == "__main__":
12     main()
```

핵심 포인트

C++의 `main()`은 언어가 정한 프로그램 진입점이지만, Python의 `main()`은 관례적인 함수 이름이다. Python에서 실제로 특별한 부분은 `if __name__ == "__main__":`이다.

3. 모듈 구조

Python에서는 하나의 .py 파일을 하나의 모듈로 사용할 수 있다. 기능을 여러 파일로 나누면 코드의 역할이 명확해지고, 다른 파일에서 필요한 기능만 불러와 재사용할 수 있다.

예를 들어 계산 기능을 calculator.py에 모아 두고, 실행 파일인 02_module_structure.py에서 import하여 사용할 수 있다.

일반적인 import 방식은 다음과 같다.

```
1 import calculator
```

이 경우 함수는 모듈 이름을 통해 호출한다.

```
1 calculator.add(10, 5)
```

특정 함수만 가져오거나 별칭을 사용할 수도 있다.

```
1 from calculator import add
2 import calculator as calc
```

학습용 파일에서는 01., 02.처럼 번호를 붙여 관리하지만, 실제로 import할 모듈 파일은 calculator.py, analysis.py, data.py처럼 역할이 드러나는 이름을 사용하는 것이 좋다.

3.1. 모듈 코드: calculator.py

```
1 def add(a, b):
2     return a + b
3
4
5 def subtract(a, b):
6     return a - b
7
8
9 def multiply(a, b):
10    return a * b
11
12
13 def divide(a, b):
14    return a / b
```

3.2. 최종 코드: 02_module_structure.py

```
1 import calculator
2
3
4 def main():
5     print(calculator.add(10, 5))
6     print(calculator.subtract(10, 5))
7     print(calculator.multiply(10, 5))
8     print(calculator.divide(10, 5))
9
10
11 if __name__ == "__main__":
12     main()
```

핵심 포인트

모듈 분리의 핵심은 기능과 실행을 분리하는 것이다. 한 파일은 계산, 데이터 처리, 분석과 같은 기능을 제공하고, 다른 파일은 그 기능을 import하여 전체 프로그램을 구성할 수 있다.

4. Data Pipeline

Data Pipeline은 데이터가 프로그램 안에서 어떤 순서로 이동하고 처리되는지를 나타내는 흐름이다. 작은 프로그램이라도 입력, 처리, 분석, 출력의 역할을 분리하면 전체 구조를 이해하기 쉬워진다.

예를 들어 학생 점수 분석 프로그램은 다음과 같은 흐름을 가진다.

```
load data
  ↓
analyze data
  ↓
display result
```

각 단계를 하나의 함수로 분리하면 함수는 하나의 역할에 집중할 수 있다. 데이터 로딩 함수는 데이터를 반환하고, 분석 함수는 계산 결과를 반환하며, 출력 함수는 전달받은 결과를 화면에 표시한다.

4.1. 최종 코드: 03_data_pipeline.py

```
1 def load_data():
2     return [80, 95, 70, 100, 85]
3
4
5 def analyze_data(scores):
6     average = sum(scores) / len(scores)
7     highest = max(scores)
8     lowest = min(scores)
9
10    return average, highest, lowest
11
12
13 def display_result(average, highest, lowest):
14     print(f"Average: {average}")
15     print(f"Highest: {highest}")
16     print(f"Lowest: {lowest}")
17
18
19 def main():
20     scores = load_data()
21
22     average, highest, lowest = analyze_data(scores)
23
24     display_result(average, highest, lowest)
25
26
27 if __name__ == "__main__":
28     main()
```

핵심 포인트

프로젝트에서는 함수 하나가 여러 일을 동시에 하기보다 역할을 나누는 것이 좋다. `main()`은 각 함수를 순서대로 호출하며 전체 데이터 흐름을 조립하는 역할을 담당한다.

5. Error Handling Integration

예외 처리는 단순히 `try-except` 문법을 사용하는 것에서 끝나지 않는다. 프로그램을 함수로 나눈 뒤에도 어느 위치에서 오류를 처리할지 결정해야 한다.

사용자가 여러 숫자를 입력하고 평균을 계산하는 프로그램에서는 숫자가 아닌 값을 입력할 수 있고, 아무 값도 입력하지 않을 수도 있다.

- 숫자가 아닌 값을 정수로 변환하면 `ValueError`가 발생한다.
- 빈 리스트의 평균을 계산하면 0으로 나누게 되어 `ZeroDivisionError`가 발생한다.

함수들은 각자의 역할에 집중하게 두고, `main()`에서 전체 프로그램 실행 중 발생하는 예외를 처리하면 구조가 명확해진다.

5.1. 최종 코드: 04_error_handling.py

```

1 def get_numbers():
2     text = input("Enter numbers separated by spaces: ")
3     return [int(x) for x in text.split()]
4
5
6 def calculate_average(numbers):
7     return sum(numbers) / len(numbers)
8
9
10 def main():
11     try:
12         numbers = get_numbers()
13         average = calculate_average(numbers)
14
15         print(f"Average: {average}")
16
17     except ValueError:
18         print("Please enter numbers only.")
19
20     except ZeroDivisionError:
21         print("No numbers were entered.")
22
23
24 if __name__ == "__main__":
25     main()

```

핵심 포인트

예상 가능한 오류는 가능한 한 구체적인 예외 타입으로 처리하는 것이 좋다. 오류가 발생하면 현재의 정상 실행 흐름은 중단되고 일치하는 `except` 블록으로 이동한다.

6. Project Structure

프로그램이 커지면 모든 코드를 하나의 파일에 작성하는 방식은 관리하기 어려워진다. 이때 역할에 따라 파일을 나누면 코드의 책임이 명확해지고 수정과 재사용도 쉬워진다.

Day 10에서는 학생 점수 분석 프로그램을 다음과 같이 세 파일로 나눈다.

```

day10/
├── 05_main.py
├── analysis.py
└── data.py

```

`data.py`는 데이터를 제공하고, `analysis.py`는 분석 기능을 제공하며, `05_main.py`는 두 모듈을 불러와 전체 실행 흐름을 관리한다.

6.1. 데이터 모듈: data.py

```
1 def load_scores():
2     return [80, 95, 70, 100, 85]
```

6.2. 분석 모듈: analysis.py

```
1 def calculate_average(scores):
2     return sum(scores) / len(scores)
3
4
5 def find_highest(scores):
6     return max(scores)
7
8
9 def find_lowest(scores):
10    return min(scores)
```

6.3. 최종 코드: 05_main.py

```
1 import data
2 import analysis
3
4
5 def main():
6     try:
7         scores = data.load_scores()
8
9         average = analysis.calculate_average(scores)
10        highest = analysis.find_highest(scores)
11        lowest = analysis.find_lowest(scores)
12
13        print(f"Average: {average}")
14        print(f"Highest: {highest}")
15        print(f"Lowest: {lowest}")
16
17    except ZeroDivisionError:
18        print("No data available.")
19
20
21 if __name__ == "__main__":
22     main()
```

파일	역할
data.py	데이터 제공
analysis.py	데이터 분석
05_main.py	모듈을 조립하고 프로그램 실행

핵심 포인트

프로젝트 구조의 핵심은 파일을 많이 만드는 것이 아니라 각 파일의 역할을 명확하게 나누는 것이다. 실행 파일은 전체 흐름을 조립하고, 기능 파일은 특정 작업을 담당하도록 구성한다.

7. Day 10 종합 실습: Student Score Analyzer

이번 종합 실습에서는 사용자에게 점수들을 입력받고 평균, 최고점, 최저점을 계산하여 출력한다. Day 10에서 배운 함수 분리, 데이터 흐름, 예외 처리, `main()` 함수, `main guard`를 하나의 프로그램에 통합한다.

7.1. 프로그램 요구사항

1. `get_scores()`에서 공백으로 구분된 점수들을 입력받는다.
2. 입력 문자열을 정수 리스트로 변환하여 반환한다.
3. `analyze_scores(scores)`에서 평균, 최고점, 최저점을 계산한다.
4. 세 분석 결과를 반환한다.
5. `display_result()`에서 분석 결과를 화면에 출력한다.
6. `main()`에서 입력, 분석, 출력 함수를 순서대로 호출한다.
7. 숫자가 아닌 값이 입력되면 `ValueError`를 처리한다.
8. 아무 점수도 입력되지 않으면 `ZeroDivisionError`를 처리한다.
9. 마지막에 `main guard`를 사용하여 `main()`을 호출한다.

7.2. 최종 코드: `practice_day10.py`

```
1 def get_scores():
2     text = input("Enter numbers separated by spaces: ")
3     return [int(x) for x in text.split()]
4
5
6 def analyze_scores(scores):
7     average = sum(scores) / len(scores)
8     highest = max(scores)
9     lowest = min(scores)
10
11     return average, highest, lowest
12
13
14 def display_result(average, highest, lowest):
15     print(f"Average: {average}")
16     print(f"Highest: {highest}")
17     print(f"Lowest: {lowest}")
18
19
20 def main():
21     try:
22         scores = get_scores()
23         average, highest, lowest = analyze_scores(scores)
24         display_result(average, highest, lowest)
25
26     except ValueError:
27         print("Please enter numbers only.")
28
29     except ZeroDivisionError:
30         print("No scores were entered.")
31
```



```

32
33 if __name__ == "__main__":
34     main()

```

7.3. 프로그램 구조 분석

기능	사용한 개념
점수 입력과 변환	<code>input()</code> , <code>split()</code> , list comprehension
평균, 최고점, 최저점 계산	함수, <code>sum()</code> , <code>len()</code> , <code>max()</code> , <code>min()</code>
여러 값 반환	tuple unpacking
결과 출력	함수 분리, f-string
전체 실행 흐름 관리	<code>main()</code>
잘못된 숫자 입력 처리	<code>ValueError</code>
빈 입력 처리	<code>ZeroDivisionError</code>
직접 실행 여부 확인	main guard

핵심 포인트

Day 10의 목표는 새로운 문법을 많이 배우는 것이 아니라 지금까지 배운 문법을 역할별 함수와 파일 구조 안에 배치해 하나의 프로그램으로 구성하는 감각을 익히는 것이다.

8. 실습 파일 목록

실습

Day 10의 학습 파일은 다음과 같이 관리한다.

- 01_main_guard.py
- 02_module_structure.py
- 03_data_pipeline.py
- 04_error_handling.py
- 05_main.py
- analysis.py
- calculator.py
- data.py
- practice_day10.py
- python_day10.tex
- python_day10.pdf

9. 핵심 요약

- `__name__`은 Python이 각 모듈에 자동으로 제공하는 특별한 변수이다.
- 파일을 직접 실행하면 `__name__`의 값은 `"__main__"`이 된다.

- `if __name__ == "__main__":`은 파일을 직접 실행했을 때만 특정 코드를 실행하도록 한다.
- Python의 `main()`은 강제된 진입점이 아니라 프로그램의 주 실행 흐름을 나타내기 위한 관례적인 함수 이름이다.
- 하나의 `.py` 파일은 다른 Python 파일에서 모듈로 사용할 수 있다.
- `import`를 이용하면 다른 모듈의 함수와 클래스를 재사용할 수 있다.
- 프로젝트에서는 역할이 드러나는 모듈 이름을 사용하는 것이 좋다.
- Data Pipeline은 데이터가 입력되고 처리되며 결과로 출력되는 전체 흐름을 의미한다.
- 입력, 분석, 출력 기능을 함수로 분리하면 각 함수의 책임이 명확해진다.
- 분석 함수는 결과를 직접 출력하기보다 반환하면 다른 코드에서 재사용하기 쉽다.
- `main()`은 여러 함수를 순서대로 호출해 전체 프로그램 흐름을 조립할 수 있다.
- 예외 처리는 함수로 나눈 프로그램의 실행 흐름에도 통합할 수 있다.
- 예상 가능한 오류는 가능한 한 구체적인 예외 타입으로 처리하는 것이 좋다.
- 오류가 발생하면 현재 정상 실행 흐름은 중단되고 일치하는 `except` 블록으로 이동한다.
- 프로그램이 커지면 데이터, 분석, 실행처럼 역할별로 파일을 분리할 수 있다.
- 프로젝트 구조에서 중요한 것은 파일의 개수보다 각 파일의 역할과 책임을 명확하게 나누는 것이다.
- Day 10은 Python 문법 학습을 실제 프로그램 구조와 프로젝트 준비 단계로 연결하는 과정이다.